AI Middle Manager

A brief rundown — how the AI works, how OpenAI fits in, and how the app is run. · 18 June 2026

What it is

An **AI operations manager** that supervises frontline staff (motel housekeeping, to start). One human manager can oversee many workers: staff get a daily task list, talk to the AI by voice during their shift, and run photo quality checks. The AI checks in proactively, logs issues, verifies work, and flags what needs a person.

How the AI works (the loop)

STEP 1
Assign
Each worker gets that day's tasks (e.g. rooms to turn over).

→

STEP 2
Interact
Worker calls the AI by voice; photographs finished work.

→

STEP 3
Verify & log
AI scores the photo, records the outcome, tracks progress.

→

STEP 4
Escalate
Anything off-track is surfaced to the human manager.

It follows a few firm rules so it stays trustworthy:

- **Grounded** — every number or fact comes from a real database query, never invented.
- **Reported ≠ verified** — what a worker *says* is stored as a claim; the photo check is a separate proof.
- **Proposes, doesn't just do** — actions that change data are logged, and big changes wait for human approval.

Where OpenAI fits in the process

The AI "brain" is mostly plain logic and database queries. OpenAI is the language/perception layer — it is called at three specific moments:

1. Voice — the AI you call (OpenAI Realtime)

Worker taps **Call the AI Manager** and speaks to it live. The phone streams audio **directly to OpenAI**; our server only hands out a short-lived token. Mid-call the AI can look up tasks or log an issue.

2. Vision — the photo quality check (gpt-5.4-mini)

Worker photographs the finished room. OpenAI **describes what it actually sees**, scores each checklist item pass/fail, and rejects off-topic photos — so a pass can't be faked.

3. Text — making sense of a call

After a call, OpenAI summarises it and pulls out the key details for the manager side.

Not OpenAI: **when** to check in = plain scheduling rules on a timer (no AI); push notifications = Web Push; outbound reminder calls (manager side) = Vapi.

The moving parts

- **Worker app** — mobile web app (PWA) you add to your phone's home screen. React + Vite, hosted on Netlify.
- **Manager dashboard** — a website (Next.js) for oversight, escalations and reminders.
- **Supabase** — the shared backend: Postgres database (locked down per user), small server-side functions, and a cron timer.

How the app is run

Day to day

- **Worker:** installs the app to their home screen, signs in, sees today's tasks, taps to complete them, calls the AI anytime, and photo-checks finished rooms. Gets timed reminders (morning shift check-in, "rooms due by 2pm").
- **Manager:** opens the dashboard to watch progress, review escalations, and trigger reminders.

Keeping it running (technical)

- **Front-end changes** — build the worker app and publish it to Netlify (one command).
- **Backend / AI changes** — database, function and prompt changes go live **instantly** through Supabase, no redeploy.
- **The timer** — a cron job checks every minute and sends the reminders/notifications on schedule.
- **Secrets** — the OpenAI key and push keys live in server-side function secrets, never shipped in the app.